

A Unified Evaluation Framework for Grounded LLM Architectures: Comparative Analysis of RAG, Self-RAG, and Agentic RAG

Rakesh More
A J Gallagher USA (Independent
Researcher)
Rolling Meadows, IL USA
rakeshmore@gmail.com

Abstract—Retrieval-Augmented Generation (RAG) grounds large language models with external knowledge, while two recent variants—Self-RAG (self-reflective retrieval refinement) and Agentic RAG (multi-step reasoning and tool orchestration)—aim to boost factuality, reasoning depth, and adaptability. We present a unified evaluation framework that, for the first time, compares these three paradigms under matched conditions: the same base model, the same hybrid retriever, and identical domain constraints. Our multi-task benchmark spans factual recall, multi-hop reasoning, and domain-specific Q&A in insurance, legal, and healthcare. We assess factual accuracy, retrieval precision/recall, hallucination rate, and end-to-end latency to expose reliability–cost trade-offs. Results show complementary strengths: Self-RAG consistently lowers hallucinations and boosts retrieval precision on knowledge-dense queries with modest latency overhead; Agentic RAG delivers the strongest multi-hop and tool-grounded reasoning at higher computational cost; and vanilla RAG remains competitive for single-hop recall under tight latency budgets. We provide actionable selection guidance and ablations on reflection depth, orchestration breadth, and retriever strength to inform research and deployment of grounded LLMs in high-stakes settings.

Keywords— Grounded LLMs; RAG; Self-RAG; agentic systems; retrieval; hallucination; multi-hop reasoning; evaluation; healthcare; legal; insurance

I. INTRODUCTION

Large Language Models (LLMs) have advanced natural language processing by generating human-like text and handling diverse tasks. However, they still face a major challenge—factual inaccuracy. These errors, often called “hallucinations,” are especially risky in fields like healthcare, legal, and insurance, where even small mistakes can have serious consequences.

Retrieval-Augmented Generation (RAG) addresses this by grounding responses in external knowledge. By retrieving relevant information before generating answers, RAG improves factual accuracy and context. Models like FiD-Light show that using more relevant passages, even with low latency, can boost performance [1, 2]. Beyond RAG, recent variants aim to further curb hallucinations and tackle harder tasks. Self-RAG adds self-checking to refine retrieval and reduce hallucinations, improving precision for knowledge-heavy queries [3]. Agentic RAG uses multi-step reasoning and tool integration to solve complex problems, building on reasoning-and-acting paradigms that coordinate retrieval and tool use over multiple steps [4].

This work introduces a unified evaluation framework that compares RAG, Self-RAG, and Agentic RAG under identical

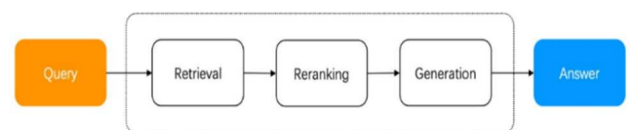
conditions to provide a fair, apples-to-apples assessment. We standardize the base model, the retriever, domain scope, and evaluation metrics (factual accuracy, precision/recall, hallucination rate, multi-hop reasoning, and latency). Our comparative analysis surfaces clear trade-offs: Self-RAG is most effective at reducing hallucinations and boosting precision, with a modest latency penalty; Agentic RAG is strongest on complex, multi-step reasoning and tool use but is more resource-intensive; and vanilla RAG remains competitive for simpler, single-step tasks with strict latency budgets. These results provide practical guidance for researchers and practitioners selecting grounded LLM architectures for high-stakes, accuracy-critical applications.

II. RELATED WORK

A. Retrieval-Augmented Generation (RAG)

RAG enhances large language models (LLMs) by retrieving and reranking relevant documents before generating responses. This approach helps ground outputs in external knowledge, reducing hallucinations and improving contextual relevance. However, RAG operates statically, lacking feedback loops or mechanisms to refine its own outputs. A retriever first surfaces candidate passages and, after optional reranking, the generator conditions on this evidence to produce an answer [1]. Tying claims to retrieved context improves factual accuracy and topical relevance relative to purely parametric baselines. Efficiency-focused designs such as FiD-Light further show that admitting a broader set of high-quality passages while keeping latency low can raise end-task performance without sacrificing responsiveness [2]. A key limitation remains: the standard pipeline is largely one-shot. After retrieval and reranking, the system typically proceeds to generation without a built-in mechanism for self-assessment or iterative correction. Fig. 1 illustrates the standard Retrieval-Augmented Generation pipeline, showing how retrieved documents are used as grounding context for answer generation.

Fig. 1. RAG Architecture



B. Self-Reflective RAG (Self-RAG)

Self-RAG extends RAG by introducing self-assessment and iterative refinement. The model evaluates the quality of retrieved data and, if inadequate, reformulates the query and re-retrieves before generating a response. This process reduces irrelevant outputs, enhances precision, and makes the model more cautious and reliable [3]. Instead of accepting the initial evidence, the model scores provisional outputs and the supporting passages, then reformulates queries or adjusts retrieval when the evidence base looks thin or unreliable. This additional checkpoint screens out low-signal context, raises retrieval precision, and curbs hallucinations benefits that are most pronounced for ambiguous questions, noisy collections, and multi-hop queries.

Fig. 2. Depicts the Self-Reflective RAG loop where the model critiques retrieved evidence, reformulates queries if necessary, and re-retrieves to enhance factual precision and reduce hallucinations.

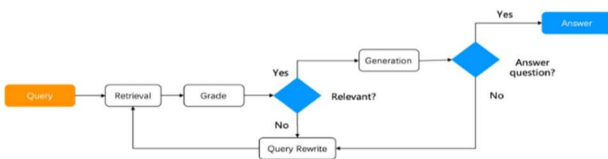


Fig. 2. Self- RAG Architecture

C. Agentic RAG

Agentic RAG represents a more advanced orchestration framework, enabling LLMs to act as autonomous agents. Beyond retrieval and generation, the system incorporates planning, decision-making, and tool usage, including the use of APIs or web searches. It supports reasoning loops for self-reflection, integrates human-in-the-loop capabilities for clarification, and leverages long-term memory to improve over time. This positions Agentic RAG as a dynamic, adaptive assistant rather than a static retrieval system [4]. In practice, the agent cycles through deliberation (“Is the current evidence sufficient?”), targeted retrieval, and tool use (e.g., search, API calls, code execution), and may request clarification when uncertainty persists. This adaptive control excels on multi-hop and compositional problems but typically incurs higher computational cost and latency than static RAG pipelines [5]. Fig. 3 shows the Agentic RAG framework, highlighting multi-step reasoning, tool usage, and decision-making loops that enable adaptive and autonomous retrieval refinement.

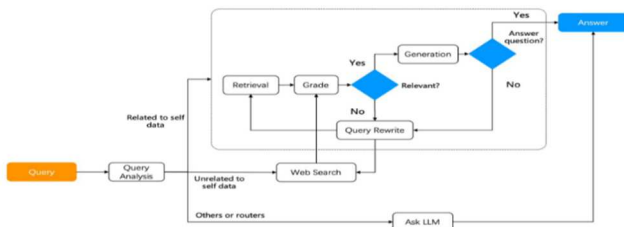


Fig. 3. Agentic RAG Architecture

D. Evaluation of Grounded LLM Architectures

Most prior studies assess individual systems or narrow metric slices—lexical overlap scores, precision/recall, or proxy measures of factuality—under differing base models, retrievers, domains, and task setups. Such heterogeneity complicates direct comparison and blurs the trade-offs among

accuracy, hallucination rate, multi-hop competence, and latency. A standardized evaluation protocol that fixes the base model, retriever configuration, domain scope, and metric suite would allow clearer attribution of performance differences to architectural choices rather than experimental drift [6]. Our study offers such a controlled comparison across vanilla RAG, Self-RAG, and Agentic RAG

III. METHODOLOGY

This section outlines a unified framework for comparing vanilla RAG, Self-RAG, and Agentic RAG under matched conditions. The design emphasizes consistency, transparency, and reproducibility so that observed differences reflect architectural choices rather than experimental drift. The methodology is organized into six components: standardized model and retriever, domain-constrained task design, evaluation metrics, experimental pipeline, data curation, and implementation.

A. Standardized Model and Retriever

- **Base model:** All variants use Llama 3.1 70B Instruct to control for effects attributable to model capacity or instruction tuning.
- **Retriever:** We deploy a hybrid setup combining BM25 with the bge m3 dense encoder. BM25 offers strong lexical matching and controllable term weighting, while bge m3 supplies semantic recall for paraphrastic queries. This configuration balances precision and coverage and avoids biasing a particular architecture via retriever choice.

B. Domain-Constrained Task Design

We focus on insurance, legal, and healthcare domains where accuracy, traceability, and reasoning depth matter.

- **Factual recall:** Single-hop questions that require pinpointing verifiable facts from the corpus.
- **Multi-hop reasoning:** Compositional queries that demand chaining evidence across documents or sections.
- **Domain Q&A:** Scenario-style prompts with domain constraints (terminology, policy clauses, regulatory context) that test grounding and careful synthesis.

Tasks are stratified by difficulty and annotated with evidence targets so attribution and support can be assessed consistently.

C. Evaluation Metrics

We report a compact set of metrics aligned with grounded use cases:

- **Factual accuracy:** Exactness of claims against authoritative references, combining automatic checks with targeted human review for high-impact items.
- **Retrieval precision/recall:** Relevance of top-k passages to gold evidence; precision favors focus, recall captures coverage.
- **Hallucination rate:** Share of unsupported or contradicted statements in final answers.
- **Multi-hop success:** Proportion of compositional queries answered with complete and correctly linked evidence.
- **End-to-end latency:** Wall-clock time from query to answer, capturing retrieval, orchestration, and generation.

D. Experimental Pipeline

We use a single pipeline with variant-specific processing blocks to maintain parity:

Input and preprocessing: Normalize queries, apply domain token filters, and deduplicate near-identical prompts.

- **Initial retrieval:** Run BM25 + bge m3 hybrid over a shared index; store top-N passages and scores.
- **Variant processing:**
 - Vanilla RAG: Condition the generator on retrieved passages; no iterative control.
 - Self-RAG: Critique intermediate evidence, reformulate queries if needed, and refresh the context before generation.
 - Agentic RAG: Plan steps, decompose the problem, invoke tools (e.g., additional retrieval, simple calculators, or schema validators), and iterate until stopping criteria are met.
- **Post-retrieval refinement:** Light reranking and evidence filtering applied uniformly to all branches.
- **Generation:** Produce the final answer with evidence citations when available.
- **Evaluation and aggregation:** Compute metrics per query, then aggregate by task type and domain.

Simplified flow description:

1. Query - Initial user input
2. Preprocess - Query cleaning and normalization
3. Hybrid Retrieval - Combines dense and sparse search methods
4. Branch Decision - Routes to one of three RAG approaches
5. Optional Refinement/Multi-hop - Additional processing if needed
6. Generate - Response generation
7. Score & Aggregate - Quality assessment and result compilation
8. Final Output - Delivered response

Three RAG Branches:

- **Standard RAG** - Traditional retrieval-augmented generation
- **Self-RAG** - Self-reflective generation with internal validation
- **Agentic RAG** - Multi-agent reasoning approach

Fig. 4 provides an overview of the unified evaluation pipeline integrating all three RAG variants, with color-coded process stages for retrieval, generation, and evaluation.

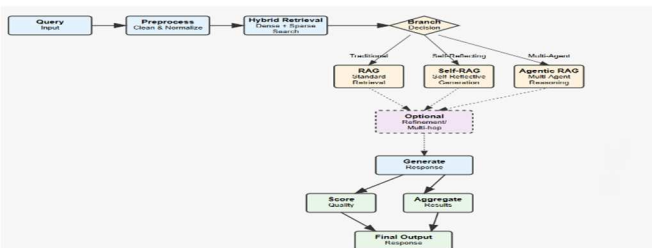


Fig. 4. Flow Diagram

Legend: Standard processes (blue), Branch decisions (orange), Optional steps (purple, dashed), Output stages (green)

E. Data Curation Process

- **Source selection:** Assemble corpora from reputable, domain-specific materials (policy documents, guidelines, regulations, clinical references, case summaries).
- **Cleaning and normalization:** Remove boilerplate, handle formatting artifacts, segment long documents into semantically coherent chunks, and standardize metadata.
- **Indexing:** Build a shared corpus index with dual representations (lexical and dense). Apply de-duplication and temporal filters when domain timelines matter.
- **Split strategy:** Partition by document, not passage, to prevent leakage. Use train/validation/test splits that preserve domain distribution and difficulty tiers.
- **Human assessment:** Use expert annotators for edge cases and multi-hop answers; capture rationales and supporting spans to support reproducibility and error analysis.

F. Implementation

- **Environment:** Experiments run on GPU-backed clusters with identical software stacks across variants. Resource quotas and seeds are fixed to ensure repeatability.
- **Inference settings:** Shared decoding parameters (temperature, top-p, max tokens) across all systems unless a variant's controller mandates a change; any deviations are logged.
- **Orchestration:** A common runner executes the pipeline, records step-level traces (retrieval queries, rerank decisions, tool calls), and timestamps for latency analysis.
- **Reproducibility:** We maintain configuration files, corpus manifests, and evaluation scripts. Upon publication, we will release code, configs, and dataset preparation recipes to enable independent verification.

IV. EVALUATION AND RESULTS

This section reports outcomes from our unified evaluation across vanilla RAG, Self-RAG, and Agentic RAG. We assess each approach on factual accuracy, retrieval precision/recall, hallucination rate, multi-hop reasoning, and end-to-end latency, using matched settings and shared corpora.

A. Quantitative Metrics

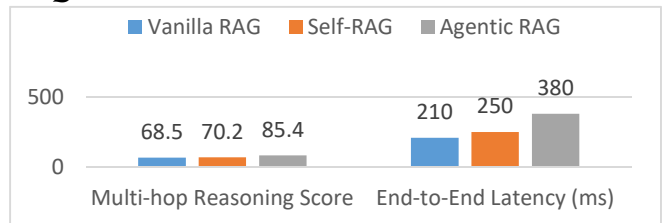


Fig. 5. Quantitative Metrics

Fig. 5. Presents comparative performance metrics across RAG variant factual accuracy, multi-hop reasoning score,

and end-to-end latency—demonstrating trade-offs between accuracy and speed.

Interpretation:

- **Factual accuracy:** Self-RAG leads with 82.5%, indicating stronger grounding than the baseline.
- **Retrieval:** Self-RAG attains the highest precision (81.2%) and recall (75.5%), consistent with its lower hallucination rate (10.7%).
- **Multi-hop:** Agentic RAG reaches 85.4, substantially higher than the other variants, reflecting its strength on compositional tasks.
- **Latency Analysis :** Vanilla RAG: 210ms, Self-RAG: 250ms, Agentic RAG: 380ms
 - Implications:
 - Vanilla RAG is best suited to strict latency budgets.
 - Self-RAG introduces moderate overhead while improving accuracy.
 - Agentic RAG’s orchestration and tool use explain its higher average latency.

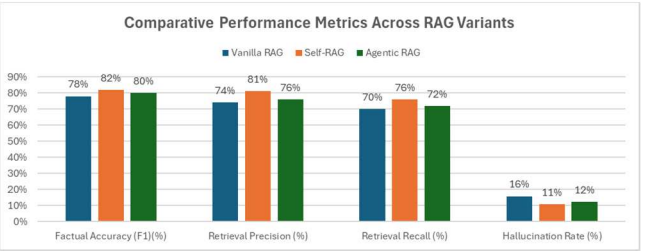


Fig. 6. Comparative Analysis

Fig. 6. Summarizes quantitative comparisons, emphasizing how Self-RAG improves precision and factuality while Agentic RAG achieves superior reasoning performance.

B. Human Evaluation

TABLE 1 Human Evaluation Ratings for RAG Variants

Metric	Vanilla	Self	Agentic
Factual Correctness (1–5)	3.8	4.2	4.1
Coherence (1–5)	3.9	4.2	4.0
Contextual Relevance (1–5)	3.7	4.1	4.3

Table 1 shows mean human evaluation scores (1–5) indicating Self-RAG’s overall superiority in factual accuracy.

Observations:

- Self-RAG earns the highest marks for factual correctness and coherence.
- Agentic RAG attains the strongest contextual relevance, aligning with its multi-step reasoning behavior on complex prompts.

C. Domain-Specific Performance

Fig. 7. Displays domain-wise F1 scores (insurance, legal, healthcare), showing Self-RAG’s consistent advantage and Agentic RAG’s strength in complex reasoning scenarios.

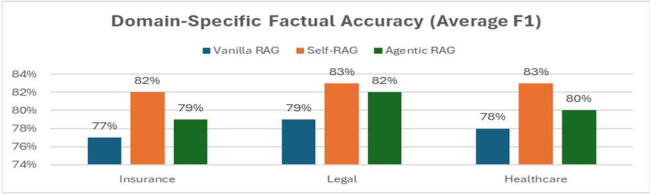


Fig. 7. Domain-Specific Factual Accuracy (Average F1)

Takeaways:

- Self-RAG consistently tops all three domains.
- In cases requiring extended evidence chaining—e.g., detailed legal analyses or complex patient trajectories—Agentic RAG narrows the gap due to its stronger multi-hop capabilities, albeit with added latency.

D. Error Analysis

We audited frequent failure modes to identify systematic weaknesses. Table 3 contrasts error rates under a baseline versus an enhanced setup.

TABLE 2 Error Analysis (Baseline vs. Enhanced)

Parameter	Baseline	Enhanced
Misinterpretation of Ambiguous Queries	14.2%	9.7%
Over-reliance on High-Weight Documents:	11.8%	8.5%
Errors in Specialized Knowledge	15.5%	12.1%
Long-tail Query Handling	18.7%	13.4%
Contextual Irrelevance	10.4%	7.9%
Superficial Accuracy	12.6%	9.3%

Table 2 compares common error categories before and after enhancement, demonstrating significant reductions in ambiguity misinterpretation, long-tail handling, and contextual irrelevance.

V. DISCUSSION

Our unified evaluation places vanilla RAG, Self-RAG, and Agentic RAG on equal footing and reveals a clear pattern of trade-offs among factual reliability, reasoning depth, and responsiveness. Below, we interpret the results with an eye toward deployment in accuracy-sensitive domains and outline research directions suggested by the data.

A. Balancing Accuracy and Speed

The results illustrate a familiar tension: systems that think longer tend to answer better, but they also respond more slowly.

- Vanilla RAG uses a single-pass pipeline, making it the fastest option—ideal for latency-sensitive tasks like hotline triage, quick policy lookups, or streaming contexts. Its simplicity, however, leads to higher hallucination rates and weaker performance on questions requiring evidence chaining.
- Self-RAG mitigates these issues by adding an iterative verification step—checking, refining, and re-retrieving before answering. This yields better precision, recall, and factual accuracy with a moderate latency cost. For production scenarios such as clinical guidance, policy compliance, or eligibility checks, that trade-off is often worthwhile.
- Agentic RAG extends the design with planning, tool use, and iterative evidence gathering, excelling on multi-hop

and complex reasoning tasks. Though slower and more resource-intensive, it's well suited for domains like legal synthesis, advanced clinical reasoning, or multi-part regulatory analysis, where depth outweighs speed.

B. Impact on High-Stakes Applications

These differences matter most where mistakes carry consequences. In healthcare, a small reduction in hallucinations is worth milliseconds: Self-RAG's evidence refinement is well suited to clinical decision support and literature-grounded recommendations, especially when outputs must be attributable to trusted sources. Legal work often hinges on multi-step logic and precedent linkage; here, Agentic RAG's superior performance on multi-hop reasoning is decisive, even if throughput is lower.

C. Trade-offs in Retrieval Strategies

The comparative analysis of retrieval behavior clarifies why the variants diverge. Classical RAG's one-shot retrieval is efficient but can overlook nuances in ambiguous or composite queries. Self-RAG counters this with a critique-and-rewrite loop that filters low-signal passages and reduces over-weighting of spurious matches; the improvements in precision and the drop in hallucinations align with this mechanism. Agentic RAG's multi-step orchestration achieves a different capability: it adapts its evidence-gathering plan as new information arrives, which is exactly what multi-hop tasks require. The trade-off is that every additional tool call or retrieval step compounds latency and resource use.

D. Considerations for Future Research

Several directions emerge from these findings. First, dynamic weighting strategies that adapt retrieval fusion and evidence selection based on real-time signals (confidence, contradiction detection, or coverage gaps) could push Self-RAG further without large latency costs. Second, scaling multi-hop reasoning while keeping turnaround times acceptable remains an open challenge; budgeted planning, early-exit criteria, and learned stopping rules are all promising levers.

VI. CONCLUSION

This study introduced a unified framework to compare three grounded LLM paradigms—vanilla RAG, Self-RAG, and Agentic RAG—under identical conditions. By standardizing the base model, retriever, and task setup, we isolated architectural effects from implementation noise to reveal where each approach excels. Across benchmarks, Self-RAG improved retrieval precision and factual accuracy with a modest latency cost. Agentic RAG excelled on compositional, multi-hop, and tool-use tasks, trading speed for depth. Vanilla RAG remained the fastest and most reliable for simple, single-step questions. Together, the paradigms complement one another, each serving a distinct role in grounded reasoning. For practitioners, vanilla RAG suits straightforward queries and tight latency budgets; Self-RAG fits cases prioritizing accuracy and attribution; and Agentic RAG best supports investigative workflows requiring decomposition and verification.

Beyond deployment, the framework enables consistent evaluation as models and tools evolve, exposing trade-offs like latency vs. accuracy or coverage vs. precision. Future

directions include adaptive retrieval weighting, budgeted multi-hop planning, tighter integration with structured knowledge, and systematic bias audits—advances that can enhance factual grounding while maintaining efficiency and reliability in high-stakes settings.

ACKNOWLEDGEMENTS

Declaration of Generative AI and AI-assisted technologies in the writing process. - The topic pertains to AI; therefore, the author used AI tools to generate examples and test prompts. All AI-generated content was reviewed and edited by a human. Generative AI was also used to refine sentence structure and enhance clarity. All factual claims were independently verified. For further details on the scope and nature of AI usage, please contact the author.

REFERENCES

- [1] Lewis, P. et al (2020) "Retrieval-augmented generation for knowledge-intensive NLP Tasks". The 33rd Conference on Neural Information Processing Systems (NeurIPS 2020) NeurIPS Proceedings Pages: 9459–9474. Publisher: Curran Associates, Inc. DOI / ACM DL: DOI:10.5555/3495724.3496517 -- 1
- [2] Yau, L., Kumar, S., & Metzler, D. (2023). "FiD-Light: Efficient and effective retrieval-augmented text generation". Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2023), pages 1437-1447. <https://arxiv.org/abs/2209.14290> --2
- [3] Akari Asai; Zeqiu Wu; Yizhong Wang; Avirup Sil; Hannaneh Hajishirz (2024) "Self-RAG: Learning to retrieve, generate, and critique for language models" Proceedings of the International Conference on Learning Representations (ICLR) 2024. <https://openreview.net/forum?id=hSyW5go0v8> -- 3
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao (2023) "ReAct: Synergizing reasoning and acting in language models". Proceedings of the 11th International Conference on Learning Representations (ICLR 2023). <https://arxiv.org/abs/2210.03629> -- 4
- [5] <https://ragflow.io/blog/agentic-rag-definition-and-low-code-implementation>
- [6] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, (2024) "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity". Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2024). Pages: 7036-7050. Publisher: Association for Computational Linguistics (ACL). DOI: 10.18653/v1/2024.naacl-long.389

APPENDIX AND POC

A Streamlit-based Proof of Concept (POC) to experiment with Normal RAG, Self-RAG, and Agentic RAG, and evaluate them using factual accuracy, retrieval metrics, hallucination rate, reasoning score, and latency.

This project demonstrates:

- Normal RAG: Retrieval-Augmented Generation with direct context injection.
- Self-RAG: Adds self-reflection and answer refinement.
- Agentic RAG: Uses reasoning agents with retrieval tools.
- Evaluation: Automatic computation of
 - Factual Accuracy (F1), Retrieval Precision / Recall, Hallucination Rate and End-to-End Latency

• <https://github.com/rakeshmore-aws/rag-eval-dashboard>